Deep Learning

Tutorial 5: Convolution Neural Network (CNN)

Submitted By: **Muhammad Mubashar**

Submitted To: **Dr Shahbaz Khan**

Reg No # ***000000494171***

Semester: **4th**

Department of Robotics & Artificial Intelligence

School of Mechanical & Manufacturing Engineering

NUST

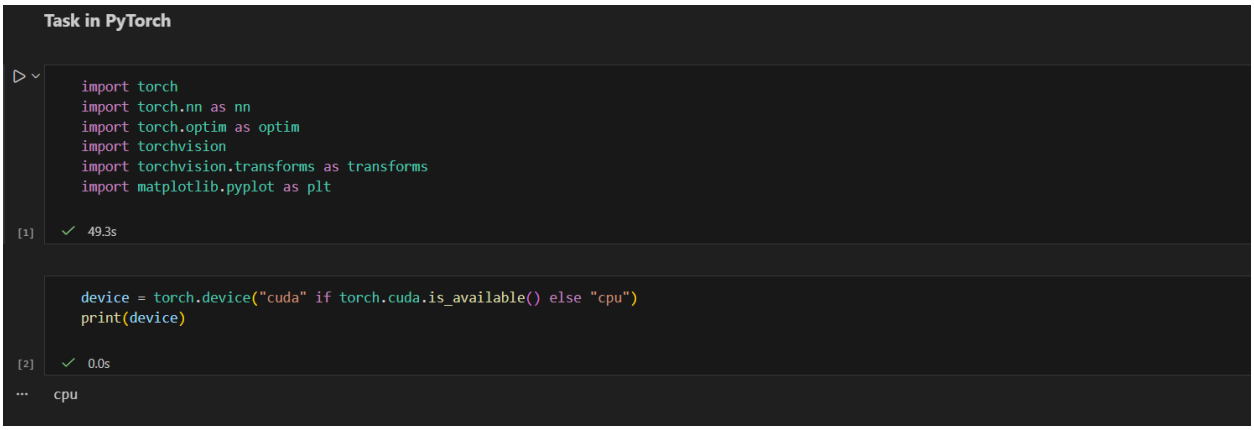
Tasks:

1. Experiment with Different Architectures
   - Modify the architecture with different filter size to see how it affects model performance.
   - Add more convolution layers or change the number of filters in each layer.
   - Add more fully connected layers or change the number of neurons in each layer and see the performance
   - Compare the performance of the modified model with the original model.

2. Improve the accuracy of the model

3. Remove the overfitting/underfitting problem

Implementation of these tasks are in *PyTorch*:



```python
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5),
                         (0.5, 0.5, 0.5))
])

train_dataset = torchvision.datasets.CIFAR10(
    root='./data', train=True,
    download=True, transform=transform)

test_dataset = torchvision.datasets.CIFAR10(
    root='./data', train=False,
    download=True, transform=transform)

train_loader = torch.utils.data.DataLoader(
    train_dataset, batch_size=64,
    shuffle=True)

test_loader = torch.utils.data.DataLoader(
    test_dataset, batch_size=64,
    shuffle=False)
```

[3] ✓ 5.0s

```python
class BasicCNN(nn.Module):
    def __init__(self):
        super(BasicCNN, self).__init__()

        self.conv_layers = nn.Sequential(
            nn.Conv2d(3, 32, 3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2, 2),

            nn.Conv2d(32, 64, 3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2, 2),

            nn.Conv2d(64, 64, 3, padding=1),
            nn.ReLU()
        )

        self.fc_layers = nn.Sequential(
            nn.Flatten(),
            nn.Linear(64 * 8 * 8, 64),
            nn.ReLU(),
            nn.Linear(64, 10)
        )

    def forward(self, x):
        x = self.conv_layers(x)
        x = self.fc_layers(x)
        return x


model = BasicCNN().to(device)
```

[4] ✓ 0.0s

```
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)
```

[5] ✓ 0.0s

```
def train_model(model, epochs=10):
    train_acc_list = []
    test_acc_list = []

    for epoch in range(epochs):
        model.train()
        correct = 0
        total = 0

        for images, labels in train_loader:
            images, labels = images.to(device), labels.to(device)

            optimizer.zero_grad()
            outputs = model(images)
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()

            _, predicted = torch.max(outputs.data, 1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()

        train_acc = 100 * correct / total
        train_acc_list.append(train_acc)

        # Evaluate
        model.eval()
```

[6] ✓ 0.0s

```
        # Evaluate
        model.eval()
        correct = 0
        total = 0

        with torch.no_grad():
            for images, labels in test_loader:
                images, labels = images.to(device), labels.to(device)
                outputs = model(images)
                _, predicted = torch.max(outputs.data, 1)
                total += labels.size(0)
                correct += (predicted == labels).sum().item()

        test_acc = 100 * correct / total
        test_acc_list.append(test_acc)

        print(f"Epoch [{epoch+1}/{epochs}] "
              f"Train Acc: {train_acc:.2f}% "
              f"Test Acc: {test_acc:.2f}%")

    return train_acc_list, test_acc_list
```

5] ✓ 0.0s

```
train_acc, test_acc = train_model(model, epochs=10)
```

[7] ✓ 23m 24.2s

```
Epoch [1/10] Train Acc: 47.84% Test Acc: 58.71%
Epoch [2/10] Train Acc: 63.31% Test Acc: 66.00%
Epoch [3/10] Train Acc: 69.90% Test Acc: 69.08%
Epoch [4/10] Train Acc: 73.93% Test Acc: 70.73%
Epoch [5/10] Train Acc: 77.29% Test Acc: 72.65%
Epoch [6/10] Train Acc: 79.65% Test Acc: 72.38%
Epoch [7/10] Train Acc: 81.91% Test Acc: 72.73%
Epoch [8/10] Train Acc: 83.78% Test Acc: 72.56%
Epoch [9/10] Train Acc: 85.92% Test Acc: 71.94%
Epoch [10/10] Train Acc: 87.75% Test Acc: 73.40%
```

TASK 1: Modify Architecture

```
class ImprovedCNN(nn.Module):
    def __init__(self):
        super(ImprovedCNN, self).__init__()

        self.conv_layers = nn.Sequential(
            nn.Conv2d(3, 64, 3, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(),
            nn.MaxPool2d(2,2),

            nn.Conv2d(64, 128, 3, padding=1),
            nn.BatchNorm2d(128),
            nn.ReLU(),
            nn.MaxPool2d(2,2),

            nn.Conv2d(128, 256, 3, padding=1),
            nn.BatchNorm2d(256),
            nn.ReLU(),
            nn.MaxPool2d(2,2)
        )

        self.fc_layers = nn.Sequential(
            nn.Flatten(),
            nn.Linear(256*4*4, 256),
            nn.ReLU(),
            nn.Dropout(0.5),
            nn.Linear(256, 10)
        )

    def forward(self, x):
        x = self.conv_layers(x)
        x = self.fc_layers(x)
        return x
```

[20]

TASK 2: Improve Accuracy

```
optimizer = optim.Adam(model.parameters(), lr=0.0005)
```

✓ 0.0s

TASK 3: Remove Overfitting

```
nn.Dropout(0.5)
nn.BatchNorm2d()
#Data Augmentation
```

```
transform = transforms.Compose([
    transforms.RandomHorizontalFlip(),
    transforms.RandomCrop(32, padding=4),
    transforms.ToTensor(),
    transforms.Normalize((0.5,0.5,0.5),
                         (0.5,0.5,0.5))
])
```

Compare Models

```
plt.plot(train_acc, label="Train Basic")
plt.plot(test_acc, label="Test Basic")
plt.legend()
plt.show()
```